Section 2 — Client-Server Communication Protocol and Binary Wire Framing

Author: Shlok Gohel (202401476), Group 103 — Member 2 Module implemented: `protocol.cpp` / `protocol.h` *(present in all four build directories)*

2.1 Why Framing Is Needed at All

TCP gives us a reliable, ordered **byte stream** — not a message stream. It preserves the order of bytes but gives no guarantee whatsoever about where one application message ends and the next begins. Two failure modes follow directly from this:

1. **Splitting.** A single `send()` of a 64 KB chunk may be delivered to the peer as several `recv()` calls of arbitrary size, depending on the MSS, the congestion window, and how full the kernel socket buffer is at that instant.
2. **Coalescing.** Several small control messages sent back to back (for example `FILE_END` followed immediately by `FILE_BEGIN`) may arrive inside a single `recv()` buffer, with no separator between them.

A naive implementation that assumes "one `send()` equals one `recv()`" appears to work on loopback, where buffers are large and latency is near zero, and then fails on the three-node lab testbed under load. This is precisely the class of bug the framing layer is designed to make impossible.

Why delimiters were rejected. A text-protocol approach — terminating each message with `\n` or a sentinel sequence — is unusable here because the payload is JPEG image data from the Animals-10 dataset. Binary image bytes can and do contain `0x0A`, `0x00`, and every other byte value, so any delimiter would eventually collide with real file content and desynchronise the stream. There is no byte value that is safe to reserve.

Chosen solution: length-prefixed binary frames. Every message, whether it is a 4-byte control message or a 64 KB block of image data, is preceded by a fixed-size header that states exactly how many payload bytes follow. The receiver reads the fixed header first, learns the exact payload length, and then reads exactly that many bytes. Message boundaries become explicit and independent of how TCP happens to segment the stream.

2.2 The Common 8-Byte Frame Header

Every frame in the entire system — all four cases, all fourteen opcodes — begins with the same 8-byte header in network byte order (big-endian):

Offset	Field	Type	Purpose
0–1	Magic	`uint16_t`	Protocol identifier `0x4954` ("IT"); detects framing desynchronisation
2	Version	`uint8_t`	Protocol revision, `0x01`
3	Type	`uint8_t`	Message opcode, `0x01` – `0x0E`
4–7	Payload Length	`uint32_t`	Number of payload bytes immediately following the header
8…	Payload	`uint8_t[]`	Exactly *Payload Length* bytes of serialized fields or raw file data

Table 2.1: Common 8-byte length-prefixed frame header.

The magic number is not decoration. If a bug anywhere in the system causes the reader to lose byte alignment, the very next header read will almost certainly not begin with `0x4954`, and the connection is torn down immediately with a `PROTOCOL_ERROR` rather than silently writing garbage into the output files. A corrupt transfer that fails loudly is far cheaper to debug than one that produces subtly wrong images.

A frame is rejected immediately if any of the following holds:

- `Magic != 0x4954`
- `Version != 0x01`
- `Payload Length > MAX_FRAME_PAYLOAD` (1 MB)

The 1 MB cap matters for more than tidiness. `Payload Length` is attacker- or bug-controlled input that is used directly to size a heap allocation. Without the cap, a single corrupted header claiming `0xFFFFFFFF` bytes would trigger a 4 GB allocation. Since the largest legitimate frame is a 64 KB `FILE_DATA` chunk plus 16 bytes of metadata, 1 MB is a generous bound that still rejects nonsense instantly.

2.3 Serialization Rules

To eliminate ambiguity in message encoding, all frame payloads use explicit binary serialization rather than any form of struct dumping.

1. Fixed-width big-endian integers. Every integer field is serialized as an unpadded fixed-width unsigned integer (`uint16_t` = 2 bytes, `uint32_t` = 4 bytes, `uint64_t` = 8 bytes) in network byte order via `htons()`, `htonl()` and `htobe64()`, and decoded with `ntohs()`, `ntohl()` and `be64toh()`.

2. No struct is ever placed on the wire. It is tempting to declare a payload `struct` and `memcpy()` it into the socket, but the compiler is free to insert padding bytes between members to satisfy alignment requirements, and the amount of padding depends on the compiler, its flags, and the target architecture. Two machines could therefore disagree about the byte layout of the "same" struct. Our serializers write each field individually into a byte buffer, so the wire layout is defined by our code and nothing else.

3. Length-prefixed variable strings. Every variable-length string (topic, filename, error reason) is encoded as a 2-byte big-endian length followed by exactly that many raw UTF-8 bytes, with **no null terminator**. This allows filenames containing spaces or arbitrary characters without any delimiter collision, and it means the receiver always knows in advance how much to read.

4. The 128-bit session identifier. The session identifier is a 128-bit value throughout the system. On the wire it is always a fixed 16-byte binary array (`uint8_t[16]`). In the stateless modes (Part I and Part II Case 1) all sixteen bytes are zero. The 32-character lowercase hexadecimal rendering (`char[33]` including the terminator) is used **only** when writing the client checkpoint file or a diagnostic log line, never on the socket. Keeping exactly one wire representation removes an entire category of conversion bugs; the helpers `session_id_to_hex()` and `hex_to_session_id()` are the only sanctioned bridge between the two forms.

5. Control versus data. The receiver distinguishes control messages from file-chunk payloads strictly via the header's Type byte. Opcode `0x05` (`FILE_DATA`) carries file bytes; every other opcode carries control metadata. The payload is never inspected to guess what kind of message it is.

2.4 Unified Message Opcodes and Exact Payload Layouts

Code	Message	Dir.	Exact Binary Payload Layout (big-endian)
`0x01`	GET	C→S	`[topic_len:u16][topic]`
`0x02`	GET_OK	S→C	`[session_id:16B][file_count:u32][total_bytes:u64]`
`0x03`	ERROR	S→C	`[err_code:u16][msg_len:u16][reason]`
`0x04`	FILE_BEGIN	S→C	`[file_index:u32][byte_offset:u64][bytes_following:u64][name_len:u16][filename]`
`0x05`	FILE_DATA	S→C	`[file_index:u32][offset:u64][data_len:u32][data]`
`0x06`	FILE_END	S→C	`[file_index:u32]`
`0x07`	TRANSFER_END	S→C	`[files_sent:u32][useful_bytes:u64]`
`0x08`	CHECKPOINT_ACK	C→S	`[session_id:16B][file_index:u32][ack_offset:u64]`
`0x09`	RESUME	C→S	`[session_id:16B][file_index:u32][byte_offset:u64][topic_len:u16][topic][name_len:u16][filename]`
`0x0A`	RESUME_OK	S→C	`[file_index:u32][accepted_offset:u64][rem_files:u32][rem_bytes:u64]`
`0x0B`	RESTART	S→C	`[code:u16][msg_len:u16][reason]`
`0x0C`	GET_ENHANCED	C→S	`[num_streams:u16][topic_len:u16][topic]`
`0x0D`	JOIN_SESSION	C→S	`[session_id:16B][conn_id:u32][bmap_len:u32][chunk_bitmap]`
`0x0E`	ASSIGN_WORK	S→C	`[conn_id:u32][file_index:u32][chunk_offset:u64][chunk_len:u32]`

Table 2.2: Unified protocol message opcodes and exact binary payload definitions.

Notes on specific messages:

- `ERROR` (`0x03`) and `RESTART` (`0x0B`) share an identical payload layout. They are kept as separate opcodes because they mean different things to the client: `ERROR` reports a

request that can never succeed (404 `TOPIC_NOT_FOUND`, 400 `BAD_REQUEST`, 500 `IO_ERROR`), whereas `RESTART` reports a *resume* that was rejected (410 `SESSION_EXPIRED`, 409 `TOPIC_MISMATCH` / `FILENAME_MISMATCH`, 416 `INVALID_OFFSET`) and tells the client to fall back to a fresh `GET`. The implementation shares one serializer internally while exposing both names.

- `FILE_BEGIN` carries both `byte_offset` and `bytes_following` rather than just the file size, so that a resumed transfer can announce "starting at byte 262144, 786432 bytes remain" without the client having to recompute it.
- `FILE_DATA` repeats `file_index` and `offset` in every chunk. This is eight extra bytes per 64 KB chunk — roughly 0.02% overhead — and it is what makes the Case 2 Enhanced design possible, because chunks arriving out of order across four parallel connections are self-describing and can be written straight to the correct offset with `pwrite()`.
- `JOIN_SESSION` uses a `uint32_t` bitmap length rather than the usual `uint16_t` string prefix, because a large topic can need more than 65535 bytes of chunk bitmap.

2.5 Implementation Structure

The layer is implemented in `protocol.cpp` / `protocol.h` as three stacked levels, each of which hides a specific class of error from the layer above.

Level 1 — Loop-complete socket helpers

```
ssize_t send_all(int sockfd, const void *buf, size_t len);
ssize_t recv_all(int sockfd, void *buf, size_t len);
```

Both loop until exactly `len` bytes have been transferred, which is what makes partial `send()` / `recv()` returns a non-issue for every other module in the project. Two details matter:

- `send()` is called with `MSG_NOSIGNAL`. Without it, writing to a socket whose peer has already closed raises `SIGPIPE`, whose default action terminates the process. Since Part II deliberately drops connections at a configurable rate, the server would otherwise kill itself the first time a fault fired.
- A `recv()` or `send()` interrupted by a signal (`errno == EINTR`) is retried rather than reported as an error. This keeps the `SIGINT` / `SIGTERM` graceful shutdown handler from corrupting an in-flight frame.

`recv_all()` distinguishes two cases that look identical to a careless implementation: if the peer closes before *any* byte of the requested read has arrived, that is a clean end of stream and `0` is returned; if the peer closes *part-way through*, the frame is unrecoverable and `-1` is returned.

Level 2 — Frame send and receive

```
int send_frame(int sockfd, uint8_t opcode, const uint8_t *payload, uint32_t
payload_len);
int recv_frame(int sockfd, FrameHeader *out_hdr, std::vector<uint8_t>
&out_payload);
```

`send_frame()` builds the 8-byte header on the stack, writes it with `send_all()`, then writes the payload with a second `send_all()`. `recv_frame()` reads the header, validates magic/version/length, resizes the output vector to the exact payload length, and reads the body.

`recv_frame()` returns one of four values, and the distinction between them drives the entire fault-tolerance design:

Return	Name	Meaning	Caller's action
`1`	`FRAME_OK`	Complete valid frame	Parse and dispatch on the opcode
`0`	`FRAME_EOF`	Peer closed cleanly, nothing pending	Transfer complete, or peer quit
`-1`	`FRAME_SOCK_ERR`	Connection dropped mid-stream	Enter the reconnect/resume path
`-2`	`FRAME_PROTO_ERR`	Bad magic, version, or oversized length	Close; do not retry

Table 2.3: `recv_frame()` return codes.

The separation of `FRAME_EOF` from `FRAME_SOCK_ERR` is the single most important contract this module exposes. An injected fault closes the socket in the middle of a transfer, which surfaces as `FRAME_SOCK_ERR`; a transfer that ended normally surfaces as `FRAME_EOF` after `TRANSFER_END`. Case 1's bounded retry loop and Case 2's `RESUME` logic both branch on exactly this distinction, so collapsing the two would silently break recovery in both cases.

Level 3 — Per-opcode serializers and parsers

Each opcode has a matched `serialize_*` / `parse_*` pair implementing its row of Table 2.2, built on a small set of internal big-endian primitives (`put_u16`, `put_u32`, `put_u64`, `put_string`, and a bounds-checked `Reader` for decoding).

Every parser enforces a strict rule: it returns `true` only if the payload decoded cleanly **and** every byte was consumed. A payload that is too short is rejected, and so is one with trailing bytes. This closes the over-read that would otherwise occur if a length field inside the

payload claimed more data than the frame actually carried — the bounds check fails and the caller treats the result exactly like a `PROTOCOL_ERROR`.

A thin convenience wrapper (`send_get()`, `send_file_data()`, …) combines serialization and framing into a single call, so no other module in the project ever constructs a payload buffer by hand.

2.6 Case-by-Case Protocol Usage

Part I (single- and multi-threaded). Opcodes `0x01`–`0x07`, with a zeroed 16-byte `session_id`. Sequence:

```
GET → GET_OK → (FILE_BEGIN → FILE_DATA × k → FILE_END) per file →
TRANSFER_END
```

Part II Case 1 (no session management). Same opcodes `0x01`–`0x07`, `session_id` still zero. On any mid-transfer failure the client reconnects and sends a fresh `GET`, and the server restarts the topic from file 0, byte 0. No protocol extension is required — the fault tolerance comes entirely from the client's retry loop.

Part II Case 2 (with session management). Opcodes `0x01`–`0x0B`. `GET_OK` now carries a real random 128-bit `session_id`. After a fault the client sends `RESUME` (`0x09`) with its last durable checkpoint; the server replies `RESUME_OK` (`0x0A`) with a validated `accepted_offset`, or `RESTART` (`0x0B`) to force a clean fallback to `GET`.

Part II Case 2 Enhanced (parallel TCP). Opcodes `0x01`–`0x0E`. The primary connection sends `GET_ENHANCED` (`0x0C`) and receives the file manifest, while *N* parallel sockets attach via `JOIN_SESSION` (`0x0D`) and pull dynamically assigned 64 KiB work units via `ASSIGN_WORK` (`0x0E`) followed by `FILE_DATA` (`0x05`).

A single framing implementation serves all four cases. Part I simply never emits opcodes above `0x07`; the unused serializers are present but never called. This was a deliberate choice over maintaining four trimmed variants, since four copies of the same logic would inevitably drift apart during integration.

2.7 Verification

The framing layer was developed and validated independently of every other module, before integration began. The test harness (`test_protocol.cpp`) replaces the real TCP connection with a local `socketpair(AF_UNIX, SOCK_STREAM)`, writes messages into one end and decodes them from the other, and asserts bit-exactness field by field. **154 assertions pass with zero failures.** The suite covers:

- Round-trip of all fourteen opcodes, every field compared against the value sent.

- A full 64 KB `FILE_DATA` chunk filled with a byte pattern that includes `0x00` and `0xFF`, verified byte-for-byte — the direct proof that binary image data survives the encoding.
- Inspection of the raw bytes on the wire, confirming the header is literally `49 54 01 06 00 00 00 05` for a 5-byte `FILE_END` frame — i.e. genuinely big-endian rather than accidentally correct on a little-endian host.
- A filename containing a space, confirming length-prefixed strings are immune to delimiter collision.
- Malformed headers (wrong magic, wrong version, a declared 2 MB payload) each rejected with `FRAME_PROTO_ERR`, and `send_frame()` refusing to emit a payload above 1 MB.
- A clean `close()` reported as `FRAME_EOF`, versus a valid header followed by a truncated body reported as `FRAME_SOCK_ERR` — the exact byte sequence produced by a mid-chunk fault injection.
- Parsers rejecting truncated payloads, payloads with trailing bytes, and a `FILE_DATA` frame whose internal `data_len` claims more bytes than the frame contains.
- Session-id helpers: zeroing, hex rendering, and hex round-trip.

The module compiles with `g++ -O2 -Wall -Wextra -pthread -std=c++17` with zero warnings in all four build directories, and the full suite also passes under AddressSanitizer and UndefinedBehaviorSanitizer with no reported memory errors or undefined behaviour.